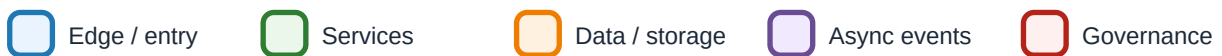


Used Car Dealer SaaS Platform

1.1 E2E Architecture Diagram - Request Flow + Async/Event Flow

This PDF contains an implementation-oriented E2E architecture diagram for the MVP microlith design: Dealer Service, Vehicle Service, and Search Service. It captures public website routing, authenticated dealer/admin flows, JWT and tenant resolution, synchronous entitlement validation, transactional outbox, Pub/Sub event delivery, search read-model updates, and future async consumers such as notification, analytics, and AI enrichment.

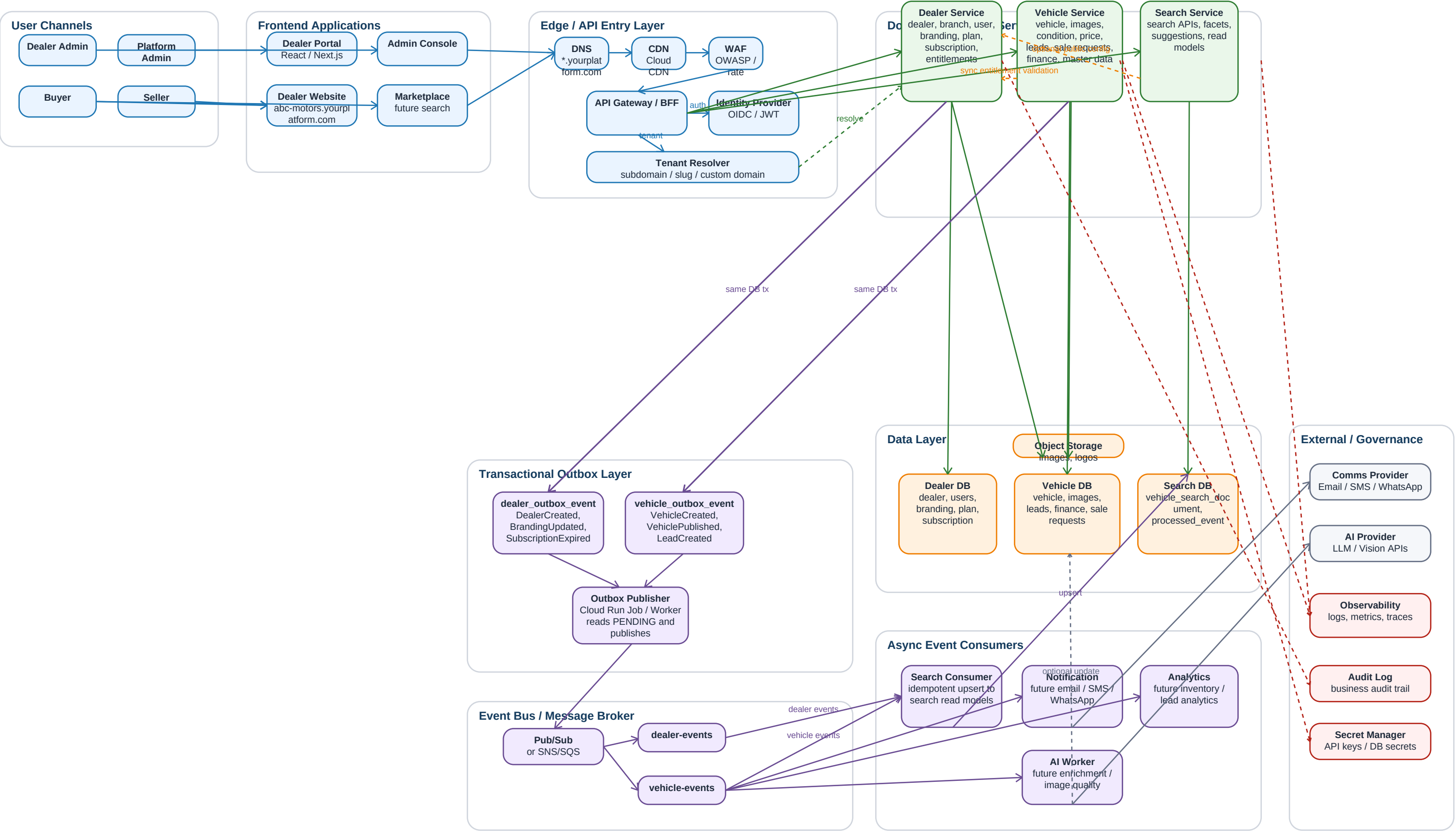
Legend



Purpose for GitHub Copilot / AI systems

AI/code-generation context: Use this diagram to guide GitHub Copilot prompts. Search updates are eventually consistent. Dealer Service and Vehicle Service write outbox events in the same database transaction as business changes. Search Service consumes events and updates denormalized read models with idempotent upserts. Avoid distributed transactions.

1.1 E2E Architecture Diagram - Request Flow + Async/Event Flow



Key architecture rules for implementation:

1. Public dealer website resolves dealer by subdomain, slug, or custom domain before search APIs are called.
2. Authenticated dealer operations use dealerId from JWT/TenantContext; every tenant-owned query must be scoped by dealerId.
3. Vehicle Service may call Dealer Service synchronously for entitlements; Search indexing is asynchronous only.
4. Dealer Service and Vehicle Service write business data + outbox event in one local database transaction.
5. Outbox Publisher publishes events to Pub/Sub; consumers are idempotent and use processed_event or equivalent logic.
6. Search Service updates denormalized read models and should not mutate transactional vehicle/dealer tables.

AI / GitHub Copilot Context Summary

Services and ownership

Dealer Service owns dealer, branch, dealer user, branding, plan, subscription, and entitlement APIs.

Vehicle Service owns vehicle inventory, image metadata, condition reports, price history, leads, seller sale requests, finance enquiries, and vehicle master data.

Search Service owns denormalized read models and search APIs. It should not own transactional vehicle/dealer data.

Request-flow rules

1. Frontend calls API Gateway/BFF.
2. Public dealer website resolves dealer context from subdomain, slug, or custom domain.
3. Authenticated dealer APIs use JWT and TenantContext to enforce dealer isolation.
4. Vehicle Service may call Dealer Service synchronously only when immediate validation is required, such as checking dealer active status, subscription, listing limit, or feature entitlements.

Async/event-flow rules

1. Business services do not directly call Search Service for indexing.
2. Dealer Service and Vehicle Service write outbox_event rows in the same database transaction as business state changes.
3. Outbox Publisher reads PENDING events and publishes them to Pub/Sub or SNS/SQS.
4. Search Service consumes events and upserts vehicle_search_document and dealer_search_document.
5. Pub/Sub delivery is at-least-once; all consumers must be idempotent.
6. Avoid distributed transactions and two-phase commit.

Recommended events

Dealer events: DealerCreated, DealerUpdated, DealerSuspended, BrandingUpdated, SubscriptionActivated, SubscriptionExpired.

Vehicle events: VehicleCreated, VehicleUpdated, VehiclePublished, VehicleStatusChanged, VehicleSold, VehicleDeactivated, VehiclePriceChanged, VehicleImageAdded, VehicleImageRemoved, ConditionReportUpdated, LeadCreated, SaleRequestCreated, FinanceEnquiryCreated.

Copilot instruction

When generating code, always check tenant isolation, outbox transaction boundaries, idempotent consumer processing, and clear service ownership boundaries.